Name: M Bharath

Roll no:2303A52060

Batch no:39

Task Description -1 (Refactoring – Eliminating Repetitive Logic)

• Task: Use AI-assisted coding techniques to restructure a Python

program that performs repeated calculations, transforming it into

a reusable and maintainable design.

Instructions:

• Ask AI to locate repeated computations within the program.

• Encapsulate repeated logic into reusable functions.

• Verify that program output remains unchanged after refactoring.

• Add proper docstrings to all defined functions

Prompt:

```
#Refactor the following Python code by removing repeated calculations.
# the logic into a reusable function that calculates area and perimeter of a
rectangle.
#Ensure the output remains the same and add proper docstrings
```

Code:

```
def rectangle_details(length, width):
    """
    Calculate and print the area and perimeter of a rectangle.

    Parameters:
    length (int): Length of rectangle
    width (int): Width of rectangle
```

```
    """
    area = length * width
    perimeter = 2 * (length + width)

    print("Area:", area)
    print("Perimeter:", perimeter)
```

Output:

```
C:\Users\dell\OneDrive\Ai Assitant>C:
neDrive/Ai Assitant/block_chain_ass/A
Area: 32
Perimeter: 24
```

Task Description -2 (Refactoring – Modularizing Inline

Calculations)

• Task: Use AI to refactor a Python script by extracting repeated

inline calculations into reusable and well-documented functions.

Instructions:

• Identify repeated calculation patterns in the code.

• Convert them into reusable functions.

• Ensure readability and proper documentation.

Prompt:

```
#Refactor the following Python code by extracting repeated inline calculations
into a reusable function.
# Ensure proper documentation and readability.
```

Code:

```python
def calculate_final_amount(amount):
    """
    Calculate the final amount after applying 10% discount.

    Parameters:
    amount (float): Original amount

    Returns:
    float: Final amount after discount
    """
    discount = amount * 0.10
    final_amount = amount - discount
    return final_amount


print("Final Amount:", calculate_final_amount(1000))
print("Final Amount:", calculate_final_amount(2000))
```

output:

```
Area: 72
Perimeter: 36
Final Amount: 900.0
Final Amount: 1800.0
```

Task Description -3 (Refactoring – Improving Loop and

Conditional Performance)

• Task: Apply AI-based suggestions to enhance the performance of

Python code that relies on nested loops and repeated conditional

checks.

Instructions:

• Use AI to recommend more efficient data structures or logic

simplifications.

• Replace inefficient constructs while preserving program behavior.

• Compare execution efficiency before and after refactoring.

Prompt:

```
#Optimize the following Python code by improving lookup performance.
#Replace nested loops with a more efficient data structure while keeping the
output same.
```

Code:

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

# Convert list to set for faster lookup
student_set = set(students)

for name in check_list:
    if name in student_set:
        print(name, "found")
    else:
        print(name, "not found")
```

output:

```
Charan found
Esha not found
Bhavya found
```

Description -4 (Refactoring – Replacing Hardcoded Values

with Constants)

• Task: Utilize AI assistance to improve maintainability by

replacing hardcoded numerical values with named constants.

Instructions:

• Detect hardcoded numeric values used directly in expressions.

• Define descriptive constants at the beginning of the program.

• Update calculations to use these constants without changing

Results.

Prompt:

```
#Refactor the following Python code by replacing hardcoded numeric values with
named constants.
#Define constants at the beginning of the program.Constants
```

Code:

```
RATE = 2
TIME = 5

def calculate_simple_interest(principal):
    """
    Calculate simple interest.

    Parameters:
    principal (int): Principal amount

    Returns:
    float: Simple interest
    """
    return (principal * RATE * TIME) / 100


print("Simple Interest:", calculate_simple_interest(1000))
print("Simple Interest:", calculate_simple_interest(2000))
```

Output:

```
Simple Interest: 100.0
Simple Interest: 200.0
```

Task Description -5 (Re factoring – Enhancing Variable Naming

and Code Clarity)

• Task: Apply AI tools to improve code readability by renaming

unclear variables and adding meaningful comments..

Instructions:

• Replace non-descriptive variable names with meaningful

identifiers.

• Add inline comments where the logic may be unclear.

• Ensure program output remains the same

Prompt:

```
#Improve the readability of the following Python code by renaming unclear
variables and adding comments.
#Ensure that the output remains unchanged
```

Code:

```
 Store two numbers
first_number = 50
second_number = 30

# Calculate their sum
total_sum = first_number + second_number

# Print the result
print(total_sum)
```

output:

```
80
```